

AI/ML INTERNSHIP PROJECT DEMONSTRATION

Project Title

AI/ML Job Salary Prediction

Submitted By:	Vismay Vinod
Course:	Bachelor of Computer Applications (BCA)
Semester:	5th / 6th Semester (as applicable)
Internship Area:	Artificial Intelligence and Machine Learning
Project Type:	Machine Learning Regression Project
Academic Year:	2026–2027
College Name:	[College Name]
University Name:	[University Name]
Department:	[Department Name]
Internship Organization:	[Internship Organization]
Faculty / Guide Name:	[Guide Name]
Date of Submission:	[DD / MM / YYYY]

Notebook: AI_ML_Salary_Prediction.ipynb | Dataset: ai_ml_job_analysis.csv

Repository: <https://github.com/Vismay-dev1/AI-ML-Salary-Prediction>

CERTIFICATE

This is to certify that **Vismay Vinod**, a student of **Bachelor of Computer Applications (BCA)** at **[College Name]**, **[University Name]**, has carried out the internship project entitled “**AI/ML Job Salary Prediction**” as part of the **Artificial Intelligence and Machine Learning** internship at **[Internship Organization]** for a duration of **[Internship Duration]**, under the guidance of **[Guide Name]**.

The work reported in this document is based on the student’s own Jupyter Notebook (AI_ML_Salary_Prediction.ipynb) and the dataset ai_ml_job_analysis.csv. To the best of my knowledge, the project has been completed satisfactorily and is submitted for evaluation.

Item	Details
Student Name	Vismay Vinod
Course	Bachelor of Computer Applications (BCA)
College	[College Name]
Internship Organization	[Internship Organization]
Project Title	AI/ML Job Salary Prediction
Internship Duration	[Internship Duration]
Guide Name	[Guide Name]

Faculty / Project Guide
[Guide Name]

Head of Department
[Department Name]

Internship Organization
[Authorised Signatory]

Date: [DD / MM / YYYY] Place: [Place] Official Seal: [Seal]

Note: This page is a certificate format with placeholders. It becomes valid only after it is completed, signed and sealed by the authorised persons of the college and the internship organization.

DECLARATION

I, **Vismay Vinod**, a student of Bachelor of Computer Applications (BCA) at [College Name], hereby declare that the project work entitled “**AI/ML Job Salary Prediction**” submitted in this report is a record of work carried out by me as part of the internship / project requirements of the BCA programme, in the area of Artificial Intelligence and Machine Learning.

The programs, outputs and graphs presented in this report were produced by running my project code (AI_ML_Salary_Prediction.ipynb) on the dataset ai_ml_job_analysis.csv. Standard open-source libraries and their public documentation have been used and are acknowledged in the references. This work has not been submitted to any other university or institution for the award of any degree or diploma.

Signature of the Student
Vismay Vinod

Date: [DD / MM / YYYY] Place: [Place]

ACKNOWLEDGEMENT

I express my sincere gratitude to **[College Name]** and the **Department of [Department Name]** for providing the academic environment and facilities required to complete this internship project.

I am thankful to my faculty / project guide, **[Guide Name]**, for the guidance, suggestions and encouragement given during the work. I also thank **[Internship Organization / Training Provider]** for the opportunity to learn the practical workflow of Artificial Intelligence and Machine Learning through this internship.

I am grateful to my teachers, classmates and friends who supported me with discussions and feedback, and to my family for their constant motivation. I also acknowledge the open-source communities behind Python, NumPy, Pandas, Matplotlib, Seaborn and Scikit-learn, whose tools and documentation made this project possible.

Vismay Vinod

ABSTRACT

This project, **AI/ML Job Salary Prediction**, applies supervised machine learning to estimate the annual salary (in USD) of an AI/ML-related job from its characteristics. Salary estimates help job seekers, students and recruiters form realistic expectations, and the project shows how data-driven models can handle many job factors together.

The project uses the dataset **ai_ml_job_analysis.csv**, which contains **500 records and 11 columns**: job title, years of experience, education, skill count, industry, company, location, remote-work type, certification and salary. The work follows a standard workflow in Python: data loading and inspection, exploratory data analysis with Pandas, Matplotlib and Seaborn, preprocessing, an 80:20 train-test split, feature scaling and model training.

In preprocessing, Education is ordinal-encoded, Remote Work is one-hot encoded, and Job Title, Industry, Company, Location and Certification are converted to numeric labels with LabelEncoder. Four regression models from Scikit-learn are trained and compared: Linear Regression, Decision Tree Regressor, Random Forest Regressor and Gradient Boosting Regressor. They are evaluated with Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and the R^2 score, and further analysed with a predicted-versus-actual plot, residual analysis and Random Forest feature importance.

On the 100-record test set used in this project, the measured R^2 values were 0.9353 (Gradient Boosting), 0.8529 (Random Forest), 0.8280 (Decision Tree) and 0.2770 (Linear Regression). A final prediction cell estimates the salary for a new job profile entered as a Python dictionary. The purpose of the project is educational: to demonstrate the complete practical machine-learning workflow for a regression problem. The predictions are estimates for this dataset and are not salary guarantees.

TABLE OF CONTENTS

CERTIFICATE	2
DECLARATION	3
ACKNOWLEDGEMENT	4
ABSTRACT	5
1. INTRODUCTION	9
1.1 Introduction to Artificial Intelligence	9
1.2 Introduction to Machine Learning	9
1.3 Machine Learning in Salary Prediction	9
1.4 Problem Statement	9
1.5 Objectives	9
2. EXISTING SYSTEM	10
3. PROPOSED SYSTEM	11
4. SYSTEM REQUIREMENTS	12
4.1 Hardware Requirements	12
4.2 Software Requirements	12
5. TECHNOLOGIES AND LIBRARIES USED	13
5.1 Python	13
5.2 NumPy	13
5.3 Pandas	13
5.4 Matplotlib	13
5.5 Seaborn	13
5.6 Scikit-learn	14
6. DATASET DESCRIPTION	15
7. IMPLEMENTATION: PROGRAMS AND OUTPUTS	16
PROGRAM 1 — LOAD DATASET	17

PROGRAM 2 — LOAD AND DISPLAY DATA	18
PROGRAM 3 — DATASET SHAPE AND INFORMATION	20
PROGRAM 4 — CHECK MISSING VALUES	22
PROGRAM 5 — EXPLORATORY DATA ANALYSIS	23
PROGRAM 6 — DATA PREPROCESSING	32
PROGRAM 7 — FEATURE AND TARGET SEPARATION	35
PROGRAM 8 — TRAIN-TEST SPLIT	36
PROGRAM 9 — FEATURE SCALING	37
PROGRAM 10 — MODEL TRAINING	39
PROGRAM 11 — MODEL EVALUATION	41
PROGRAM 12 — MODEL COMPARISON	43
PROGRAM 13 — PREDICTED VS ACTUAL GRAPH	45
PROGRAM 14 — RESIDUAL ANALYSIS	47
PROGRAM 15 — FEATURE IMPORTANCE	49
PROGRAM 16 — NEW SALARY PREDICTION	51
8. PROJECT WORKFLOW	54
9. RESULTS AND DISCUSSION	55
9.1 Dataset Observations	55
9.2 Model Performance	56
9.3 Predicted vs Actual and Residuals	57
9.4 Feature Importance	57
9.5 New Salary Prediction	58
9.6 Overall Discussion	58
10. ADVANTAGES	58
11. LIMITATIONS	58
12. FUTURE ENHANCEMENTS	59
13. CONCLUSION	59
14. LEARNING OUTCOMES	60

15. VIVA QUESTIONS AND ANSWERS	61
16. REFERENCES	64
17. APPENDIX — COMPLETE SOURCE CODE	65

1. INTRODUCTION

1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science that builds systems able to perform tasks that normally need human intelligence, such as learning from experience, recognising patterns, understanding language, making decisions and solving problems. Modern AI is used in search engines, recommendation systems, voice assistants, medical diagnosis support, fraud detection and many other applications.

1.2 Introduction to Machine Learning

Machine Learning (ML) is a part of AI in which a computer learns patterns from data instead of being programmed with fixed rules. A learning algorithm is given example data, finds relationships in it and builds a model, which can then make predictions for new data. ML is commonly divided into supervised learning (data with known answers), unsupervised learning (data without answers) and reinforcement learning (learning from rewards). This project uses **supervised learning**, specifically **regression**, because the target (salary) is a continuous number.

1.3 Machine Learning in Salary Prediction

A salary depends on several factors at the same time: the role, experience, education, skills, industry, employer, location and working arrangement. A machine-learning model can learn from past records how these factors are related to salary and then estimate the salary of a new profile. Because it uses many features together and applies the same learned rules every time, it gives consistent estimates, which can support salary benchmarking and career planning.

1.4 Problem Statement

To develop a machine learning system that predicts the salary of an AI/ML-related job based on job characteristics such as experience, education, skills, industry, company, location, remote-work type and certification.

1.5 Objectives

1. Load and understand the dataset.

2. Perform exploratory data analysis.
3. Preprocess categorical and numerical data.
4. Prepare features and target variables.
5. Split the dataset into training and testing sets.
6. Train multiple regression algorithms.
7. Evaluate the models using MAE, RMSE and R^2 .
8. Compare model performance.
9. Analyse feature importance.
10. Predict salary for a new job profile.

2. EXISTING SYSTEM

In the traditional approach, salary expectations are formed manually: by asking colleagues, reading job advertisements, browsing salary web sites or relying on a recruiter's experience. This approach has several limitations:

- **Manual estimation:** the estimate depends on a person's judgement and on the few examples that person has seen.
- **Difficulty handling many factors:** it is hard to weigh experience, education, skills, location, industry and company together in one's head.
- **Lack of consistent prediction:** two people can give different estimates for the same profile, and the same person may give different estimates at different times.
- **Difficulty in identifying relationships:** manual methods cannot easily measure how strongly each factor is related to salary or whether the relationship is linear or non-linear.
- **Time-consuming:** collecting and comparing many records by hand is slow.

Simple rules of thumb (for example, "salary rises with experience") are useful, but they cannot capture interactions between several factors. This does not mean manual methods are useless; they remain a good source of context. The proposed system adds a data-driven, repeatable estimate on top of them.

3. PROPOSED SYSTEM

The proposed system is a machine-learning pipeline built in a Jupyter / Google Colab notebook. It reads the job dataset, inspects and visualises it, converts all columns into numeric form, splits the data into training and testing parts, trains four regression models, evaluates them with standard metrics, compares them and finally predicts the salary of a new job profile. The workflow is shown in the figure below.

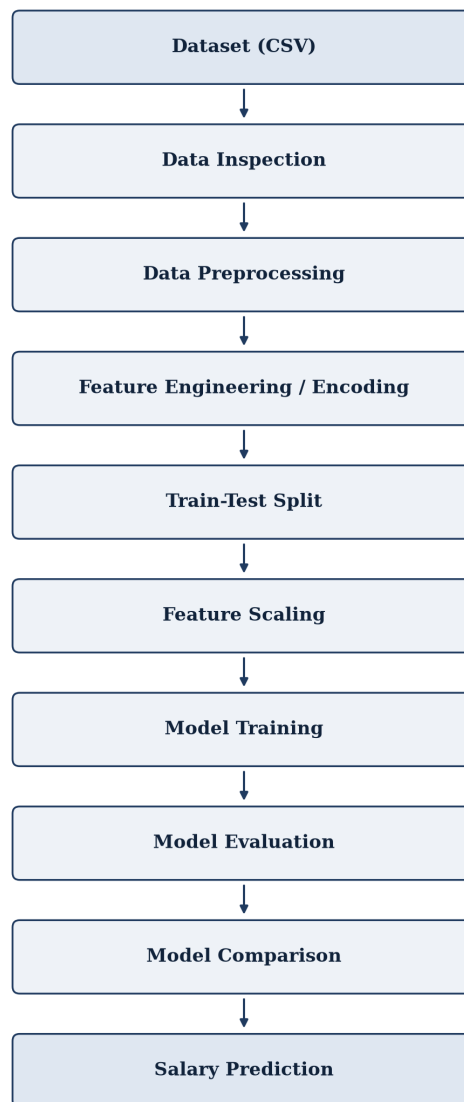


Figure 1: Workflow of the Proposed System

Each stage of the workflow is implemented as one or more cells in the notebook, and each is explained with its program and output in Chapter 7.

4. SYSTEM REQUIREMENTS

4.1 Hardware Requirements

The dataset is small (about 50 KB) and the models train in a few seconds, so the requirements are modest. The figures below are typical estimates for running the notebook; they were not obtained from a formal benchmark.

Table 1: Hardware requirements

Component	Minimum (to run the notebook)	Recommended (comfortable use)
Computer	Laptop / desktop PC	Laptop / desktop PC
Processor	Dual-core, 64-bit CPU	Quad-core CPU (Intel i5 / Ryzen 5 or better)
RAM	4 GB	8 GB or more
Storage	1 GB free disk space	5 GB free (SSD preferred)
Display	1366 × 768	Full HD (1920 × 1080)
Network	Not needed after set-up (Colab needs internet)	Broadband (for Google Colab / package install)

4.2 Software Requirements

Table 2: Software requirements

Software	Purpose	Version used for the outputs in this report
Python 3	Programming language	3.11.2
Jupyter Notebook / Google Colab	Environment to run the notebook	The README recommends Colab; the outputs here were produced by running the notebook code with Python
NumPy	Numerical computing	2.4.6
Pandas	Data handling and preprocessing	2.3.3
Matplotlib	Plotting	3.11.2
Seaborn	Statistical visualisation	0.13.2
Scikit-learn	Machine learning	1.9.1

*Operating system: Windows, Linux or macOS (any system that can run Python 3 and Jupyter).
Results may differ slightly if other library versions are used.*

5. TECHNOLOGIES AND LIBRARIES USED

5.1 Python

Python is a high-level, general-purpose programming language with simple, readable syntax. It is widely used for data science and machine learning because it has a large ecosystem of libraries, an active community and works well with notebooks.

5.2 NumPy

NumPy provides the n-dimensional array and fast mathematical operations on whole arrays. In this project it is used for numeric work such as selecting numeric columns (`np.number`) and computing the square root for RMSE (`np.sqrt`). Pandas and Scikit-learn are built on NumPy arrays.

5.3 Pandas

Pandas provides the DataFrame, a table-like structure for loading CSV files, inspecting data (`head`, `info`, `describe`), finding missing values (`isnull`), mapping and encoding columns (`map`, `get_dummies`) and separating features from the target. All dataset handling and preprocessing in this project is done with Pandas.

5.4 Matplotlib

Matplotlib is the basic plotting library of Python. It is used to create and control figures, axes, titles, labels and scatter plots, for example the predicted-versus-actual plot and the model-comparison bar charts.

5.5 Seaborn

Seaborn is built on Matplotlib and gives attractive statistical plots with little code. In this project it is used for the histograms with density curve, the correlation heatmap, box plots, the experience–salary scatter plot and the feature-importance bar chart.

5.6 Scikit-learn

Scikit-learn is the main machine-learning library of the project. It provides preprocessing tools (LabelEncoder, StandardScaler), data splitting (train_test_split), regression algorithms (LinearRegression, DecisionTreeRegressor, RandomForestRegressor, GradientBoostingRegressor) and evaluation metrics (mean_absolute_error, mean_squared_error, r2_score), all with a uniform fit() / predict() interface.

6. DATASET DESCRIPTION

The dataset **ai_ml_job_analysis.csv** (stored in the project repository) was inspected directly. It contains **500 rows (records)** and **11 columns**, which agrees with the repository README. Each row is one AI/ML job record and the column **Salary (USD)** is the target to be predicted.

Table 3: Structure of ai_ml_job_analysis.csv

Column	Description	Type	Values observed
ID	Unique record identifier	Numeric	1 – 500 (500 unique)
Job Title	Job position / role	Categorical	15 different titles
Experience (Years)	Years of professional experience	Numeric	0.0 – 20.0
Education	Education level	Categorical	Bachelors, Masters, PhD
Skill Count	Number of technical skills listed	Numeric	3 – 25
Industry	Industry sector of the job	Categorical	10 sectors
Company	Employer name	Categorical	20 companies
Location	Job location (city, country, or “Remote”)	Categorical	15 locations
Remote Work	Remote / Hybrid / No status	Categorical	Yes, No, Hybrid
Certification	Professional certification held	Categorical	7 certificates + “None” (166 rows)
Salary (USD)	Salary in US dollars (target variable)	Numeric	8,000 – 216,300

The column types were verified by running `df.info()` (see Program 3). Seven columns are text (object) columns and four are numeric: ID, Experience (Years), Skill Count and Salary (USD).

Important observation: in the Certification column, the text “None” (meaning no certification) appears in 166 rows. Pandas read_csv treats the string “None” as a missing value by default, so `df.isnull()` reports 166 missing values in Certification (Program 4). These are records without a certification, not data-entry errors. In the notebook, `astype(str)` turns them into the label “nan”, which `LabelEncoder` then encodes like any other category.

7. IMPLEMENTATION: PROGRAMS AND OUTPUTS

This chapter presents the actual programs from the notebook `AI_ML_Salary_Prediction.ipynb`, each followed by its output and a short explanation. The code is copied from the notebook without modification.

How the outputs were obtained

The notebook file in the repository was saved without cell outputs. Therefore the code cells were executed, unchanged and in order, on the real dataset, and the printed results and graphs were captured. Printed text and tables are shown as images rendered from those captured results; graphs are the actual figures drawn by the notebook code. No result in this report was typed in manually. Environment: Python 3.11.2, pandas 2.3.3, NumPy 2.4.6, scikit-learn 1.9.1, Matplotlib 3.11.2, Seaborn 0.13.2. Results in another environment (for example Google Colab with other library versions) may differ slightly.

Where the notebook itself prints nothing (for example, feature scaling), a short “supplementary verification” snippet is shown separately and clearly marked. Such snippets are not part of the notebook; they only display the state created by the notebook code.

PROGRAM 1 — LOAD DATASET

Program

```
import os

# Upload dataset in Google Colab (skips automatically if file already
# present)
if not os.path.exists('ai_ml_job_analysis.csv'):
    try:
        from google.colab import files
        print("Please upload 'ai_ml_job_analysis.csv'")
        uploaded = files.upload()
    except ImportError:
        pass # not running in Colab

assert os.path.exists('ai_ml_job_analysis.csv'), "Dataset not found.
Please upload ai_ml_job_analysis.csv"
print("Dataset ready ✅")
```

Output



```
Dataset ready ✅
```

Figure 2: Output of the Dataset Availability Check

Explanation

What it does: the program checks whether the file `ai_ml_job_analysis.csv` is present in the working directory. If it is missing and the notebook is running in Google Colab, it opens the Colab file-upload dialog; outside Colab the `ImportError` is ignored. The `assert` statement stops the notebook with a clear message if the file is still not found, and finally the message “Dataset ready” (with a green tick) is printed.

Why it is required: every later cell reads this file. Checking early gives a clear error message instead of a confusing failure later. In this run the file was already present, so no upload was needed.

PROGRAM 2 — LOAD AND DISPLAY DATA

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

df = pd.read_csv('ai_ml_job_analysis.csv')
df.head()
```

Output

	ID	Job Title	Experience (Years)	Education	Skill Count
0	1	Research Intern	2.2	PhD	10
1	2	Machine Learning Engineer	11.8	Bachelors	3
2	3	AI Consultant	13.0	Masters	16
3	4	Applied Scientist	3.2	Bachelors	13
4	5	NLP Engineer	16.9	Masters	11

	Industry	Company	Location	Remote Work
0	Technology	Infosys	Toronto, Canada	Yes
1	Finance	Wipro	Mumbai, India	Hybrid
2	Technology	Cognizant	London, UK	No
3	Automotive	Infosys	Mumbai, India	No
4	Healthcare	Cognizant	Seattle, USA	Yes

	Certification	Salary (USD)
0	Azure AI Engineer	35800
1	TensorFlow Developer	34400
2	IBM Data Science Cert	143200
3	NaN	24700
4	Microsoft Certified: Azure Data Scientist	170200

Figure 3: Dataset Preview – output of df.head() (11 columns shown in three parts for readability)

Explanation

What it does: the cell imports NumPy, Pandas, Matplotlib and Seaborn, sets the Seaborn style to “whitegrid” and the default figure size to 10×6 inches.

`pd.read_csv('ai_ml_job_analysis.csv')` reads the CSV file into a DataFrame named `df`. `df.head()` displays the first five rows (index 0 to 4).

What `head()` does: it returns the first `n` rows of a DataFrame (`n = 5` by default). It is used for a quick look at the column names, the kind of values and the format of the data.

In the notebook, `df.head()` is shown as one wide table. Here it is split into three parts only so that the text stays readable on an A4 page; the rows and values are exactly the same.

PROGRAM 3 — DATASET SHAPE AND INFORMATION

Program

```
print("Shape:", df.shape)
df.info()
```

Output

```
Shape: (500, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   ID                    500 non-null   int64
 1   Job Title             500 non-null   object
 2   Experience (Years)    500 non-null   float64
 3   Education             500 non-null   object
 4   Skill Count          500 non-null   int64
 5   Industry              500 non-null   object
 6   Company              500 non-null   object
 7   Location              500 non-null   object
 8   Remote Work          500 non-null   object
 9   Certification         334 non-null   object
10   Salary (USD)         500 non-null   int64
dtypes: float64(1), int64(3), object(7)
memory usage: 43.1+ KB
```

Figure 4: Output of df.shape and df.info()

Explanation

Number of rows and columns: df.shape returned (500, 11), i.e. 500 rows and 11 columns.

Data types: df.info() lists each column with its non-null count and data type: ID, Skill Count and Salary (USD) are int64, Experience (Years) is float64 and the other seven columns are object (text).

Non-null counts: all columns have 500 non-null values except Certification, which has 334 (explained in Program 4).

Memory information: df.info() reported “memory usage: 43.1+ KB”, the approximate memory used by the DataFrame (the “+” means the memory of the text values is a lower estimate).

This step confirms that the file was read correctly and shows which columns need encoding.

PROGRAM 4 — CHECK MISSING VALUES

Program

```
# Missing values  
df.isnull().sum()
```

Output

```
ID                0  
Job Title         0  
Experience (Years) 0  
Education         0  
Skill Count       0  
Industry          0  
Company           0  
Location          0  
Remote Work       0  
Certification     166  
Salary (USD)      0  
dtype: int64
```

Figure 5: Output of df.isnull().sum()

Explanation

What it does: df.isnull() marks every missing cell as True; .sum() adds up the True values of each column and so gives the number of missing values per column.

Result: ten columns have 0 missing values. Certification shows 166 missing values (334 non-null + 166 = 500).

Interpretation: the original CSV contains the word “None” in 166 Certification cells, meaning the person has no certification. read_csv automatically treats “None” as missing, hence the count. These are therefore not real data gaps.

Why checking is important: most machine-learning algorithms cannot handle missing values, and missing values can bias results. Checking them before training shows whether cleaning (dropping or filling) is needed. In this project the notebook converts the column to text with astype(str) during label encoding, so these entries become the category “nan” and no row is lost.

PROGRAM 5 — EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) means studying the data with statistics and graphs before building a model. The notebook performs five EDA steps, shown below as Programs 5.1 to 5.5. (In the notebook, the statistical summary, Program 5.1, comes before the missing-value check of Program 4.)

PROGRAM 5.1 — Statistical Summary

Program

```
df.describe(include='all').T
```

Output

	count	unique	top	freq	mean
ID	500.0	NaN	NaN	NaN	250.5
Job Title	500	15	Research Intern	41	NaN
Experience (Years)	500.0	NaN	NaN	NaN	9.9344
Education	500	3	Bachelors	207	NaN
Skill Count	500.0	NaN	NaN	NaN	14.126
Industry	500	10	Manufacturing	60	NaN
Company	500	20	Startup Inc	36	NaN
Location	500	15	London, UK	40	NaN
Remote Work	500	3	Hybrid	179	NaN
Certification	334	7	TensorFlow Developer	60	NaN
Salary (USD)	500.0	NaN	NaN	NaN	89118.6

	std	min	25%	50%	75%
ID	144.4818327679989	1.0	125.75	250.5	375.25
Job Title	NaN	NaN	NaN	NaN	NaN
Experience (Years)	5.663335003929528	0.0	5.5	9.6	14.6
Education	NaN	NaN	NaN	NaN	NaN
Skill Count	6.621919936641834	3.0	9.0	14.0	20.0
Industry	NaN	NaN	NaN	NaN	NaN
Company	NaN	NaN	NaN	NaN	NaN
Location	NaN	NaN	NaN	NaN	NaN
Remote Work	NaN	NaN	NaN	NaN	NaN
Certification	NaN	NaN	NaN	NaN	NaN
Salary (USD)	52186.7184368054	8000.0	41475.0	85700.0	134250.0

	max
ID	500.0
Job Title	NaN
Experience (Years)	20.0
Education	NaN
Skill Count	25.0
Industry	NaN
Company	NaN
Location	NaN
Remote Work	NaN
Certification	NaN
Salary (USD)	216300.0

Figure 6: Output of `df.describe(include='all').T` (shown in three parts)

Explanation

What it does: `describe(include='all')` summarises every column: count, unique, top and freq for text columns and count, mean, std, min, quartiles and max for numeric columns. `.T` transposes the table so that each column of the dataset is one row.

Key observations: Salary (USD) ranges from \$8,000 to \$216,300 with a mean of \$89,118.60, a median of \$85,700 and a standard deviation of \$52,186.72. Experience (Years) ranges from 0.0 to 20.0 years (mean 9.93) and Skill Count from 3 to 25 (mean 14.13). The text columns have 15 job titles, 10 industries, 20 companies and 15 locations.

PROGRAM 5.2 — Salary Distribution

Program

```
# Target variable distribution
plt.figure(figsize=(10,5))
sns.histplot(df['Salary (USD)'], kde=True, bins=30, color='teal')
plt.title('Salary Distribution')
plt.xlabel('Salary (USD)')
plt.show()
```

Output

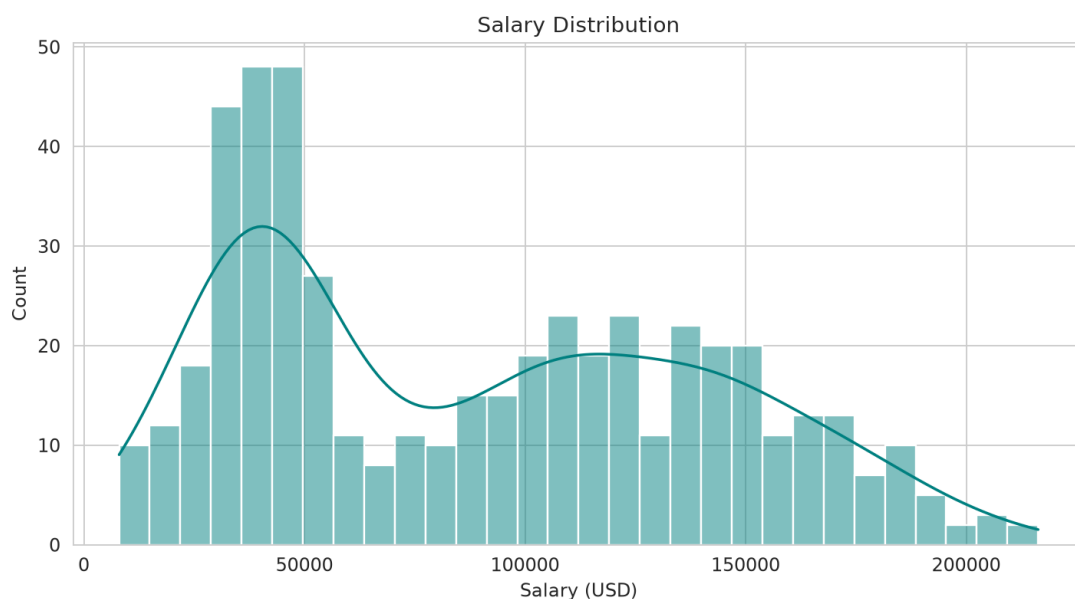


Figure 7: Salary Distribution (histogram with density curve)

Explanation

What it does: `sns.histplot` draws a histogram of the target variable with 30 bins and a smooth KDE (density) curve.

Observation: the distribution does not have a single bell shape. There is a high peak between roughly 30,000 and 55,000 USD and a second, broader group between roughly 85,000 and 190,000 USD, with few salaries above 200,000 USD. It is therefore a two-group (bimodal) distribution. Program 5.5 and Section 9.1 show what separates the groups.

PROGRAM 5.3 — Correlation Analysis

Program

```
# Numeric feature correlations
numeric_df = df.select_dtypes(include=[np.number]).drop(columns=['ID'])
plt.figure(figsize=(6,5))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap (Numeric Features)')
plt.show()
```

Output

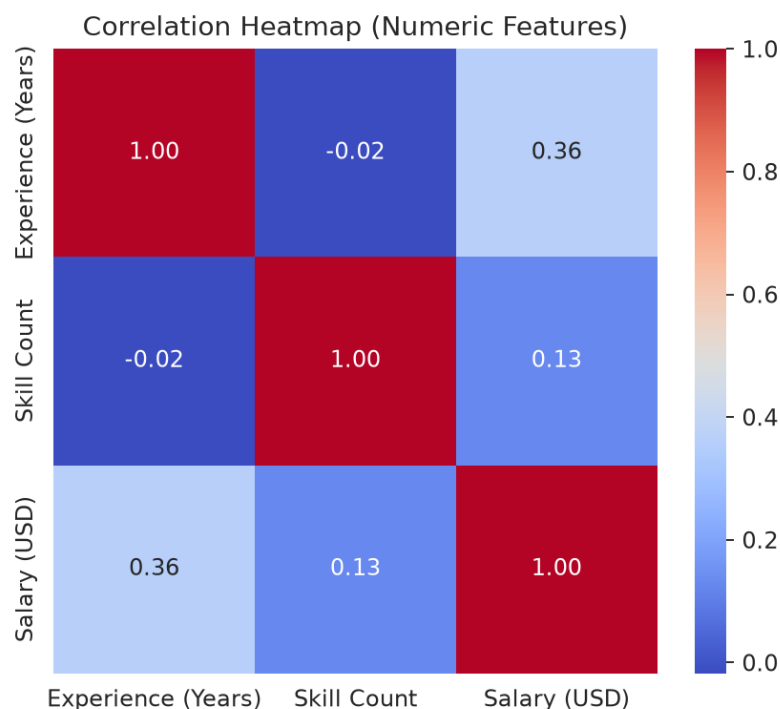


Figure 8: Correlation Heatmap (numeric features)

Explanation

What it does: the numeric columns (without ID) are selected, their correlation matrix is calculated and shown as an annotated heatmap. Correlation ranges from -1 (perfect negative) through 0 (none) to $+1$ (perfect positive) and measures linear association only.

Observation: Experience (Years) has a correlation of 0.36 with Salary and Skill Count has 0.13 . Experience and Skill Count are almost uncorrelated with each other (-0.02). Both numeric features therefore have a positive linear relationship with salary, moderate for Experience and weak for Skill Count. The other factors

are categorical and are not part of this matrix.

PROGRAM 5.4 — Salary by Categorical Features

Program

```
# Salary by categorical features
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

sns.boxplot(data=df, x='Education', y='Salary (USD)', ax=axes[0,0])
axes[0,0].set_title('Salary by Education')

sns.boxplot(data=df, x='Remote Work', y='Salary (USD)', ax=axes[0,1])
axes[0,1].set_title('Salary by Remote Work Type')

top_titles = df['Job Title'].value_counts().index
sns.boxplot(data=df, x='Job Title', y='Salary (USD)', order=top_titles,
            ax=axes[1,0])
axes[1,0].set_title('Salary by Job Title')
axes[1,0].tick_params(axis='x', rotation=75)

sns.boxplot(data=df, x='Industry', y='Salary (USD)', ax=axes[1,1])
axes[1,1].set_title('Salary by Industry')
axes[1,1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

Output

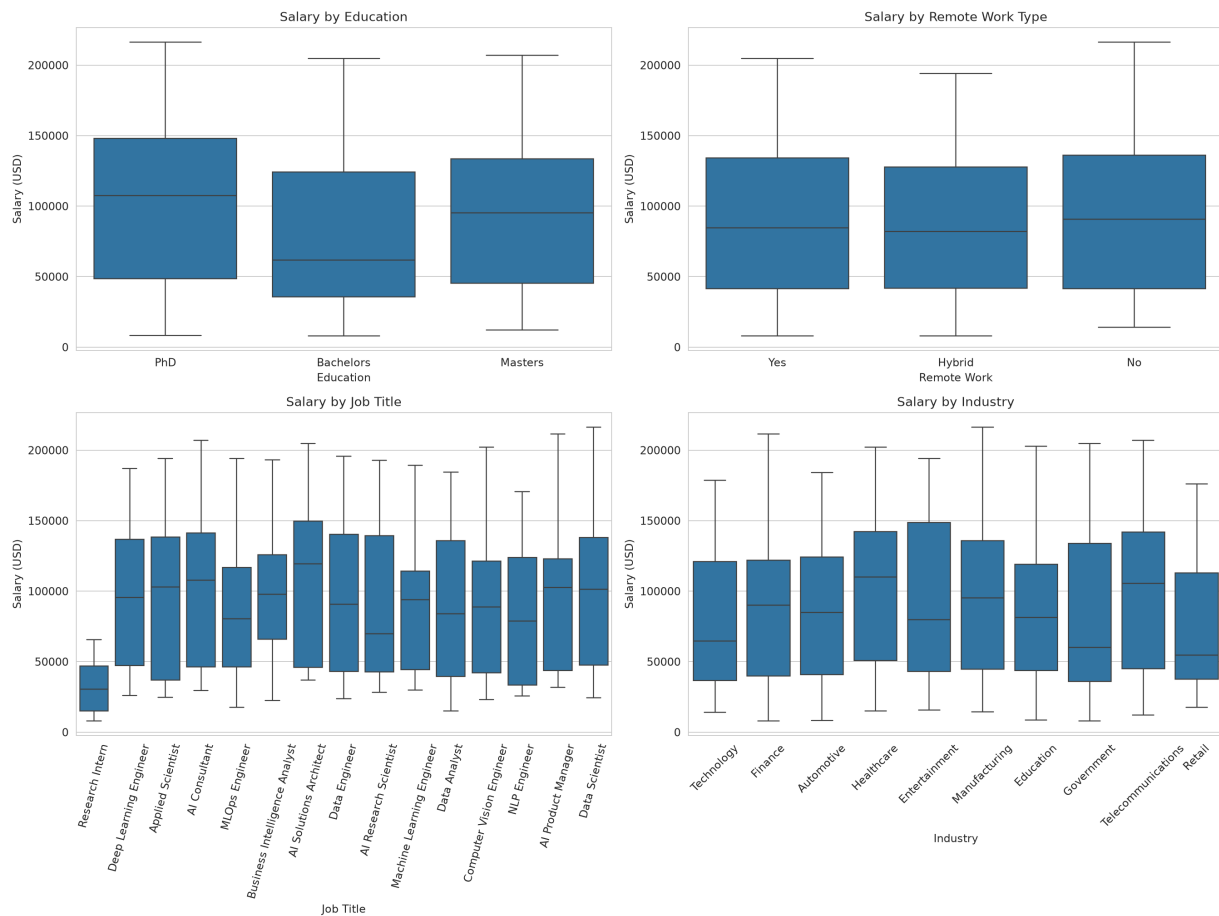


Figure 9: Salary by Education, Remote Work, Job Title and Industry (box plots)

Explanation

What it does: four box plots compare the salary distribution across Education, Remote Work type, Job Title (ordered by how often each title occurs) and Industry. Each box shows the middle 50% of salaries, the line inside is the median, and the whiskers show the range.

Observations: the median salary is lowest for Bachelors and highest for PhD, with Masters in between. The three Remote Work groups have similar medians and spreads. “Research Intern” has clearly lower salaries than all other job titles. Industries differ in their medians, but their boxes overlap a lot. In every group the salaries spread widely, which shows that no single one of these four columns explains salary alone.

PROGRAM 5.5 — Experience vs Salary

Program

```
# Experience vs Salary
plt.figure(figsize=(9,5))
sns.scatterplot(data=df, x='Experience (Years)', y='Salary (USD)',
               hue='Education', alpha=0.7)
plt.title('Experience vs Salary')
plt.show()
```

Output



Figure 10: Experience vs Salary (coloured by Education)

Explanation

What it does: a scatter plot of Experience (Years) against Salary (USD), with each point coloured by Education.

Observation: salary tends to rise with experience, which agrees with the positive correlation. The points form two separate bands: a lower band (roughly 10,000–65,000 USD) that rises slowly with experience and an upper band (roughly 60,000–215,000 USD) that is wider and rises more strongly. The Education colours are mixed inside both bands, so Education is not what separates them. Section 9.1 checks the Location column for the cause.

PROGRAM 6 — DATA PREPROCESSING

Program

```
from sklearn.preprocessing import LabelEncoder

data = df.drop(columns=['ID']).copy()

# Ordinal encode Education
education_order = {'Bachelors': 0, 'Masters': 1, 'PhD': 2}
data['Education'] = data['Education'].map(education_order)

# One-hot encode Remote Work
data = pd.get_dummies(data, columns=['Remote Work'], prefix='Remote')

# Label encode high-cardinality categorical columns
label_cols = ['Job Title', 'Industry', 'Company', 'Location',
              'Certification']
encoders = {}
for col in label_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col].astype(str))
    encoders[col] = le

data.head()
```

Output

```
Education mapping used: {'Bachelors': 0, 'Masters': 1, 'PhD': 2}
```

Figure 11: Education mapping used for ordinal encoding (supplementary print of the notebook variable `education_order`)

	Job Title	Experience (Years)	Education	Skill Count	Industry	Company
0	14	2.2	2	10	8	10
1	12	11.8	0	3	3	19
2	0	13.0	1	16	8	4
3	4	3.2	0	13	0	10
4	13	16.9	1	11	5	4

	Location	Certification	Salary (USD)	Remote_Hybrid	Remote_No
0	14	1	35800	False	False
1	7	6	34400	True	False
2	6	4	143200	False	True
3	7	7	24700	False	True
4	12	5	170200	False	False

	Remote_Yes
0	True
1	False
2	False
3	False
4	True

Figure 12: Preprocessed data – output of data.head() (12 columns shown in three parts)

Explanation

Step by step: (1) ID is dropped because it is only an identifier. (2) Education is mapped with the dictionary education_order. (3) Remote Work is one-hot encoded with pd.get_dummies. (4) The five remaining text columns are converted with LabelEncoder inside a loop, and each fitted encoder is saved in the dictionary encoders so that the same encoding can be applied to new data later.

Ordinal encoding (Education): used when categories have a natural order. Bachelors < Masters < PhD are given 0, 1 and 2, so the order is preserved.

One-hot encoding (Remote Work): each category becomes its own True/False (1/0) column: Remote_Hybrid, Remote_No and Remote_Yes. This suits unordered categories because no artificial order is created.

Label encoding: LabelEncoder converts each distinct text value of a column into an integer label (0, 1, 2, ...), assigned in alphabetical order of the values. For example, in the output the first row's Job Title "Research Intern" became 14. LabelEncoder is neither frequency encoding nor target encoding: it does not use how often a value occurs or the salary values; it only gives each category a number.

Technical clarification about the notebook: the Markdown text above this cell in the notebook describes the LabelEncoder step as "Frequency/Target-safe encoding". That wording is not correct; the code simply performs label encoding, as explained above. The code is kept unchanged. A limitation of label encoding is that the numbers imply an order between categories that does not really exist; this matters more for Linear Regression than for tree-based models.

PROGRAM 7 — FEATURE AND TARGET SEPARATION

Program

```
X = data.drop(columns=['Salary (USD)'])
y = data['Salary (USD)']

print("Feature columns:", list(X.columns))
print("X shape:", X.shape, "| y shape:", y.shape)
```

Output

```
Feature columns: ['Job Title', 'Experience (Years)', 'Education', 'Skill Count',
                  'Industry', 'Company', 'Location', 'Certification', 'Remote_Hybrid',
                  'Remote_No', 'Remote_Yes']
X shape: (500, 11) | y shape: (500,)
```

Figure 13: Output of the feature and target separation

Explanation

X = input features: all columns of the preprocessed data except Salary (USD). The 11 feature columns are Job Title, Experience (Years), Education, Skill Count, Industry, Company, Location, Certification, Remote_Hybrid, Remote_No and Remote_Yes.

y = target/output: the Salary (USD) column, the value the model must predict.

Shapes: X has shape (500, 11) – 500 records and 11 features – and y has shape (500,) – 500 salary values.

PROGRAM 8 — TRAIN-TEST SPLIT

Program

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
print(f"Train size: {X_train.shape[0]} | Test size: {X_test.shape[0]}")
```

Output

```
Train size: 400 | Test size: 100
Train share: 80% | Test share: 20%
```

Figure 14: Output of the train-test split (first line from the notebook; second line is a supplementary check)

Explanation

What it does: `train_test_split` randomly divides `X` and `y` into a training set and a testing set.

80% training / 20% testing: `test_size=0.2` reserves 20% of the data for testing, so 400 records are used for training and 100 records for testing (output: Train size: 400 | Test size: 100).

random_state=42: the split is random but fixed by the seed 42, so the same records go to the same sets every time the code is run and the results can be reproduced. The number 42 itself has no special meaning.

Why splitting is needed: a model must be tested on data it has not seen during training; otherwise the score would only show how well it memorised the training data.

PROGRAM 9 — FEATURE SCALING

Program

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Output

The notebook cell produces no printed output. The following snippet is a supplementary verification (not part of the notebook) that prints the mean and standard deviation of the scaled training data:

```
print("Mean of scaled training features (rounded):",
      X_train_scaled.mean(axis=0).round(2))
print("Std of scaled training features (rounded):",
      X_train_scaled.std(axis=0).round(2))
```

```
Mean of scaled training features (rounded): [ 0.  0. -0.  0.  0. -0. -0.  0.  0.
 0. -0.]
Std of scaled training features (rounded): [1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Figure 15: Supplementary verification of StandardScaler (mean ≈ 0 , standard deviation = 1 for each of the 11 features)

Explanation

Standardization: StandardScaler transforms each feature as $z = (x - \text{mean}) / \text{standard deviation}$. After scaling, every feature has a mean of about 0 and a standard deviation of 1, as the output above confirms for all 11 features.

Mean and standard deviation: the mean is the average value of a feature and the standard deviation shows how far values typically lie from the mean.

fit_transform and transform: the scaler learns the mean and standard deviation from the training data only (fit_transform) and then applies the same values to the test data (transform). This avoids data leakage, i.e. the test data does not influence the scaling.

Why scaling is useful: features measured on very different scales (for example experience 0–20 versus label codes 0–19) can affect algorithms that depend on

feature magnitude. Tree-based models (Decision Tree, Random Forest, Gradient Boosting) split on thresholds and generally do not need standard scaling. In this workflow the scaled data is used only for Linear Regression; the tree models use the unscaled `X_train` and `X_test`.

PROGRAM 10 — MODEL TRAINING

Program

The notebook trains and evaluates the models in one cell. Here the cell is shown in two parts: the imports with the model training code (this program) and the metric function (Program 11). The complete cell is also given in the Appendix.

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
    GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score
```

```
# --- Linear Regression (scaled) ---
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
pred_lr = lr.predict(X_test_scaled)
evaluate('Linear Regression', y_test, pred_lr)

# --- Decision Tree ---
dt = DecisionTreeRegressor(max_depth=8, random_state=42)
dt.fit(X_train, y_train)
pred_dt = dt.predict(X_test)
evaluate('Decision Tree', y_test, pred_dt)

# --- Random Forest ---
rf = RandomForestRegressor(n_estimators=300, max_depth=12,
    random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
evaluate('Random Forest', y_test, pred_rf)

# --- Gradient Boosting ---
gb = GradientBoostingRegressor(n_estimators=300, learning_rate=0.05,
    max_depth=3, random_state=42)
gb.fit(X_train, y_train)
pred_gb = gb.predict(X_test)
evaluate('Gradient Boosting', y_test, pred_gb)
```

Output

The training itself prints nothing; the printed lines are the evaluation results shown under Program 11. The following supplementary snippet (not part of the notebook) confirms that the models were fitted with the parameters in the code:

```
print("Decision Tree -> max_depth param:", dt.max_depth, "| fitted tree
      depth:", dt.get_depth(), "| leaves:", dt.get_n_leaves())
print("Random Forest -> n_estimators:", rf.n_estimators, "| max_depth
      param:", rf.max_depth, "| trees fitted:", len(rf.estimators_))
print("Gradient Boost -> n_estimators:", gb.n_estimators, "|
      learning_rate:", gb.learning_rate, "| max_depth:", gb.max_depth, "|
      stages fitted:", gb.n_estimators_)
```

```
Decision Tree -> max_depth param: 8 | fitted tree depth: 8 | leaves: 128
Random Forest -> n_estimators: 300 | max_depth param: 12 | trees fitted: 300
Gradient Boost -> n_estimators: 300 | learning_rate: 0.05 | max_depth: 3 |
      stages fitted: 300
```

Figure 16: Supplementary verification of the trained models and their parameters

Explanation

Four regression algorithms are trained on the same training data. The parameters are exactly those in the notebook.

1. Linear Regression – LinearRegression() on the scaled features. It fits a straight-line (linear) equation $\text{salary} = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots$ by minimising the sum of squared errors. It is simple and fast and serves as the baseline, but it can only represent linear relationships.

2. Decision Tree Regressor – DecisionTreeRegressor(max_depth=8, random_state=42). It splits the data again and again with yes/no questions on feature values and predicts the average salary of the training records in the final group (leaf). max_depth=8 limits the number of question levels to prevent a very deep, over-fitted tree.

3. Random Forest Regressor – RandomForestRegressor(n_estimators=300, max_depth=12, random_state=42, n_jobs=-1). It builds 300 decision trees, each on a random sample of the data and with random feature choices, and averages their predictions. Averaging reduces over-fitting and makes it more stable than a single tree. n_jobs=-1 uses all CPU cores.

4. Gradient Boosting Regressor – GradientBoostingRegressor(n_estimators=300, learning_rate=0.05, max_depth=3, random_state=42). It builds 300 small trees (depth 3) one after another; each new tree tries to correct the errors of the trees before it. learning_rate=0.05 makes each tree contribute only a small step.

PROGRAM 11 — MODEL EVALUATION

Program

This is the evaluation part of the same notebook cell. The function `evaluate()` is called after each model's `predict()` in the code of Program 10.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score

results = {}

def evaluate(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    results[name] = {'MAE': mae, 'RMSE': rmse, 'R2': r2}
    print(f"{name:22s} | MAE: {mae:10,.2f} | RMSE: {rmse:10,.2f} | R2:
        {r2:.4f}")
```

Output

```
Linear Regression      | MAE:  36,748.75 | RMSE:  42,274.27 | R2: 0.2770
Decision Tree         | MAE:  14,211.95 | RMSE:  20,621.29 | R2: 0.8280
Random Forest         | MAE:  13,886.34 | RMSE:  19,065.77 | R2: 0.8529
Gradient Boosting     | MAE:   9,090.10 | RMSE:  12,646.38 | R2: 0.9353
```

Figure 17: Output of the model evaluation (MAE, RMSE and R^2 on the test set)

Table 4: Evaluation results on the 100-record test set (taken from the program output)

Model	MAE (USD)	RMSE (USD)	R^2
Linear Regression	36,748.75	42,274.27	0.2770
Decision Tree	14,211.95	20,621.29	0.8280
Random Forest	13,886.34	19,065.77	0.8529
Gradient Boosting	9,090.10	12,646.38	0.9353

Explanation

Mean Absolute Error (MAE): the average of the absolute differences between actual and predicted salaries. In simple words, it tells by how many dollars the prediction is wrong on average. Lower is better.

Root Mean Squared Error (RMSE): the square root of the average of the squared errors. It is also in dollars, but because errors are squared first, large mistakes are penalised more. RMSE is always greater than or equal to MAE; a big gap between them means a few large errors. Lower is better.

R² score: shows how much of the variation in salary is explained by the model. 1.0 is a perfect fit, 0 is no better than always predicting the average salary, and negative values mean worse than that. Higher is better.

The notebook's `evaluate()` function computes the three metrics with `mean_absolute_error`, `mean_squared_error` (followed by `np.sqrt`) and `r2_score`, stores them in the dictionary `results` and prints one formatted line per model.

PROGRAM 12 — MODEL COMPARISON

Program

```
results_df = pd.DataFrame(results).T.sort_values('R2', ascending=False)
results_df
```

```
fig, axes = plt.subplots(1, 2, figsize=(14,5))

results_df['R2'].plot(kind='bar', ax=axes[0], color='steelblue')
axes[0].set_title('R² Score by Model')
axes[0].set_ylabel('R²')
axes[0].tick_params(axis='x', rotation=30)

results_df['RMSE'].plot(kind='bar', ax=axes[1], color='indianred')
axes[1].set_title('RMSE by Model')
axes[1].set_ylabel('RMSE (USD)')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```

Output

	MAE	RMSE	R2
Gradient Boosting	9090.101915	12646.380039	0.935301
Random Forest	13886.336010	19065.772355	0.852946
Decision Tree	14211.954351	20621.293334	0.827972
Linear Regression	36748.753686	42274.267533	0.277031

Figure 18: Model comparison table – output of results_df (sorted by R², highest first)

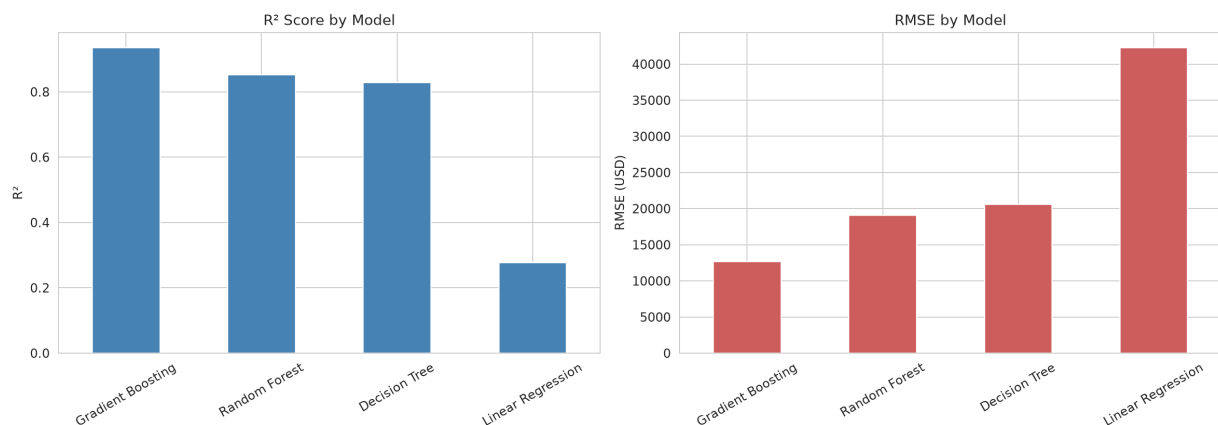


Figure 19: Model Performance Comparison (R² score and RMSE by model)

Explanation

The dictionary results is converted into a DataFrame with the models as rows (.T transposes it) and sorted by R^2 from highest to lowest. The two bar charts then show R^2 and RMSE for each model.

How the models are compared: a model is better when its MAE and RMSE are lower and its R^2 is higher. On this test set, Gradient Boosting has the lowest MAE (9,090.10) and RMSE (12,646.38) and the highest R^2 (0.9353); Linear Regression has the highest errors and the lowest R^2 (0.2770). Random Forest and Decision Tree lie between them and are close to each other (R^2 0.8529 and 0.8280). All three metrics give the same ranking in this run. Because only one random split with 100 test records was used, this ranking applies to this split; cross-validation would give a more reliable comparison.

PROGRAM 13 — PREDICTED VS ACTUAL GRAPH

Program

```
best_model_name = results_df['R2'].idxmax()
print("Best model:", best_model_name)

preds_map = {
    'Linear Regression': pred_lr,
    'Decision Tree': pred_dt,
    'Random Forest': pred_rf,
    'Gradient Boosting': pred_gb,
}
best_preds = preds_map[best_model_name]

plt.figure(figsize=(7,7))
plt.scatter(y_test, best_preds, alpha=0.6, color='darkorange')
lims = [min(y_test.min(), best_preds.min()), max(y_test.max(),
    best_preds.max())]
plt.plot(lims, lims, 'k--', label='Perfect Prediction')
plt.xlabel('Actual Salary (USD)')
plt.ylabel('Predicted Salary (USD)')
plt.title(f'Predicted vs Actual Salary – {best_model_name}')
plt.legend()
plt.show()
```

Output

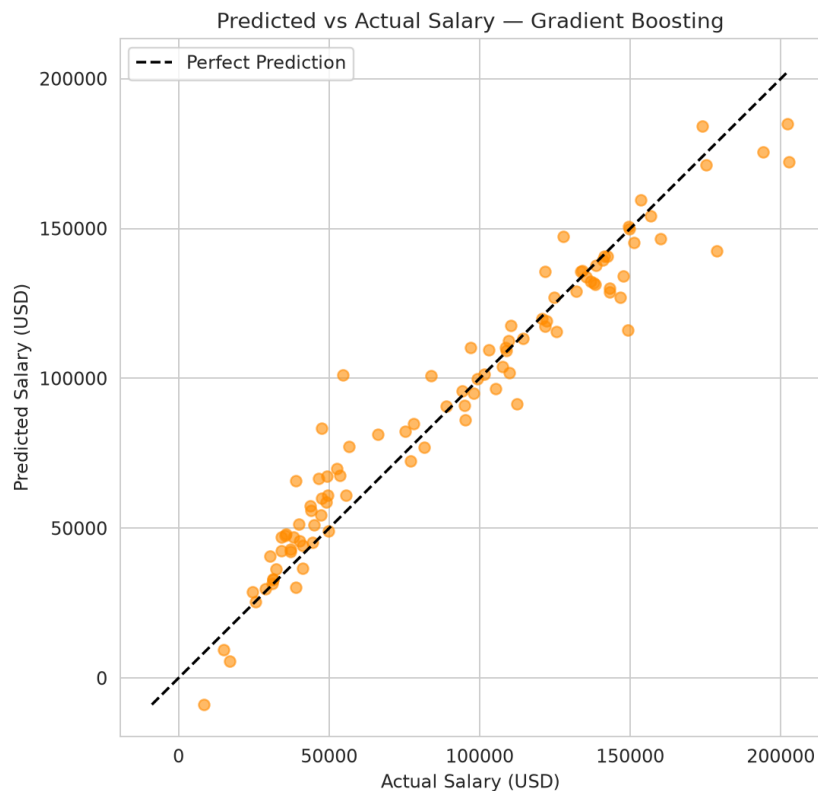


Figure 20: Predicted vs Actual Salary – Gradient Boosting

Best model: Gradient Boosting

Figure 21: Output of the best-model selection (printed line)

Explanation

What it does: `best_model_name = results_df['R2'].idxmax()` selects the model with the highest R^2 (here Gradient Boosting), takes its test predictions from `preds_map` and draws a scatter plot against the actual salaries.

X-axis = actual salary; **Y-axis** = predicted salary; the **dashed diagonal line** = perfect prediction (predicted equals actual).

Reading the graph: points closer to the diagonal are predictions closer to the actual value. In this graph most points lie near the line; the scatter is larger for some salaries above 100,000 USD, where a few points fall well below the line. One prediction (\$-8,843.02) is below zero, which is not a possible salary; Gradient Boosting can produce values outside the range of the training salaries.

PROGRAM 14 — RESIDUAL ANALYSIS

Program

```
# Residual distribution for the best model
residuals = y_test.values - best_preds
plt.figure(figsize=(9,5))
sns.histplot(residuals, kde=True, bins=30, color='purple')
plt.title(f'Residual Distribution – {best_model_name}')
plt.xlabel('Residual (Actual - Predicted)')
plt.show()
```

Output

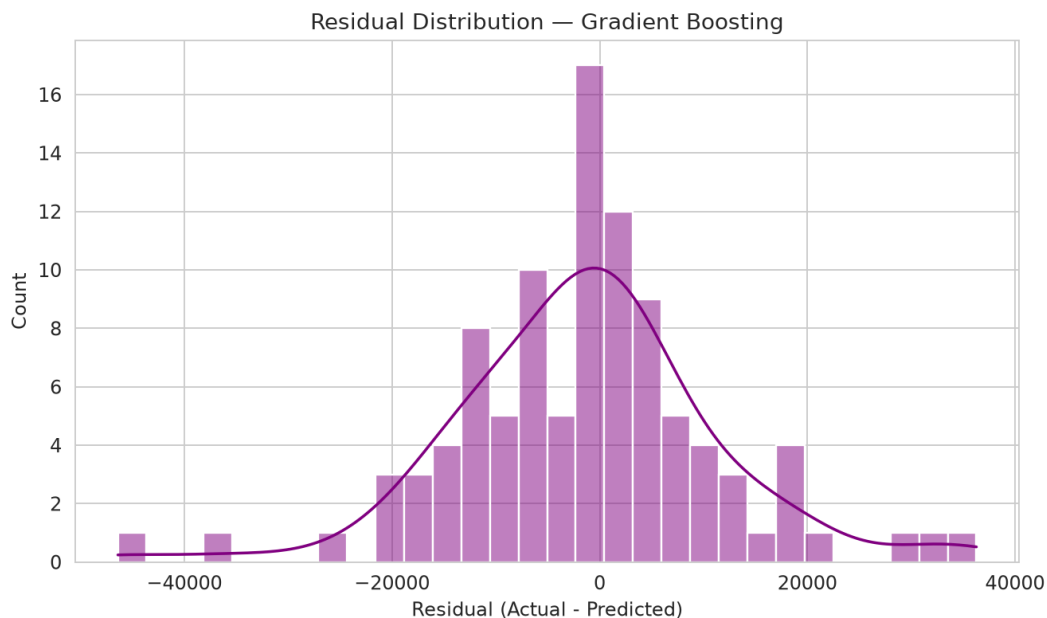


Figure 22: Residual Distribution – Gradient Boosting

Explanation

Residual = Actual – Predicted. The code computes it for the best model on the test set and plots its distribution as a histogram with a density curve. A positive residual means the model under-predicted; a negative residual means it over-predicted.

Observation: the residuals are centred close to zero: their mean is -1,415.11 USD and their standard deviation is 12,566.96 USD (calculated from the same residuals array). They range from -46,403.48 to 36,280.76 USD. 9 of the 100 test records have an absolute error above 20,000 USD; the others are within that value.

Why residual analysis is used: it checks whether errors are balanced around zero (no strong bias), shows how large the errors are and reveals unusual cases (outliers). A residual histogram that is roughly symmetric and centred on zero is a good sign.

PROGRAM 15 — FEATURE IMPORTANCE

Program

```
importances = pd.Series(rf.feature_importances_,
                        index=X.columns).sort_values(ascending=False)

plt.figure(figsize=(9,6))
sns.barplot(x=importances.values, y=importances.index, palette='viridis')
plt.title('Feature Importance — Random Forest')
plt.xlabel('Importance')
plt.show()

importances
```

Output

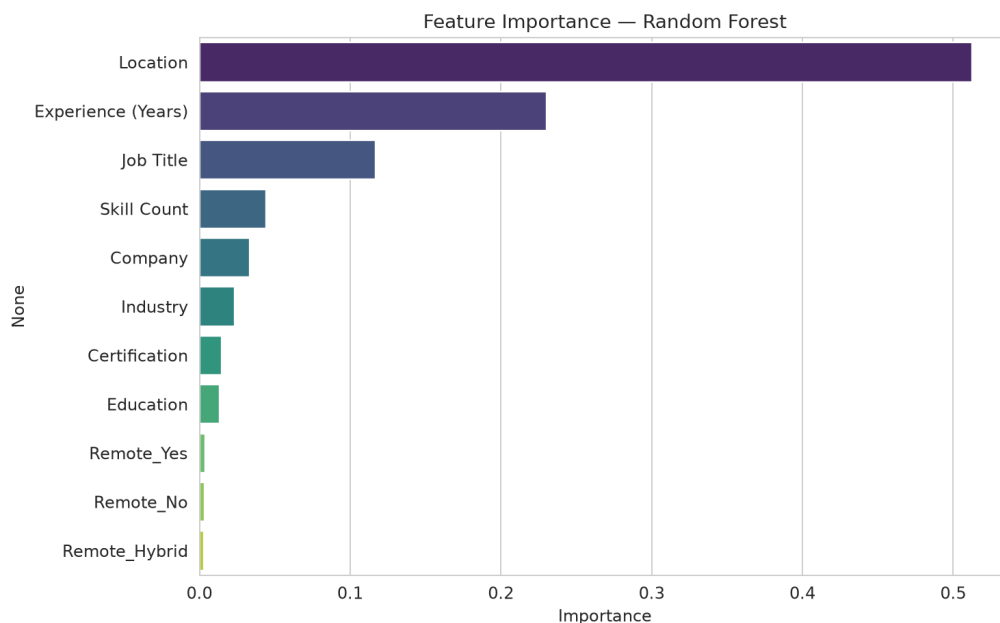


Figure 23: Random Forest Feature Importance

Location	0.512401
Experience (Years)	0.230411
Job Title	0.117060
Skill Count	0.044460
Company	0.033546
Industry	0.023223
Certification	0.014803
Education	0.013605
Remote_Yes	0.003725
Remote_No	0.003621
Remote_Hybrid	0.003146
dtype: float64	

Figure 24: Random Forest feature importance values – output of importances

Explanation

What it does: `rf.feature_importances_` gives the importance of each feature in the trained Random Forest. The values add up to 1; a larger value means the feature was used more to reduce prediction error. The values are sorted and drawn as a horizontal bar chart.

Result: Location has the highest importance (0.5124), followed by Experience (Years) (0.2304) and Job Title (0.1171). Together these three account for 86.0% of the total importance. Skill Count (0.0445), Company (0.0335) and Industry (0.0232) have smaller values, and Certification (0.0148), Education (0.0136) and the three Remote Work columns (0.0037, 0.0036, 0.0031) have the lowest importance.

Note: feature importance shows how the model used the features in this dataset; it does not prove a cause–effect relationship. Importance values of label-encoded columns with many distinct values (such as Location) can also be somewhat higher than those of columns with few values.

PROGRAM 16 — NEW SALARY PREDICTION

Program

```
new_profile = {
    'Job Title': 'Machine Learning Engineer',
    'Experience (Years)': 5.0,
    'Education': 'Masters',
    'Skill Count': 12,
    'Industry': 'Technology',
    'Company': 'Google',
    'Location': 'Bengaluru, India',
    'Remote Work': 'Yes',
    'Certification': 'AWS ML Specialty',
}

new_df = pd.DataFrame([new_profile])
new_df['Education'] = new_df['Education'].map(education_order)
new_df = pd.get_dummies(new_df, columns=['Remote Work'], prefix='Remote')

for col in label_cols:
    # unseen categories fall back to the most frequent known label
    known = set(encoders[col].classes_)
    new_df[col] = new_df[col].apply(lambda v: v if v in known else
                                    encoders[col].classes_[0])
    new_df[col] = encoders[col].transform(new_df[col].astype(str))

# align columns with training data (add any missing one-hot columns as 0)
for col in X.columns:
    if col not in new_df.columns:
        new_df[col] = 0
new_df = new_df[X.columns]

predicted_salary = rf.predict(new_df)[0]
print(f"Predicted Salary: ${predicted_salary:,.2f}")
```

Output

Predicted Salary: \$114,232.08

Figure 25: Output of the new salary prediction (Random Forest model)

Supplementary verification (not part of the notebook) that displays what was passed to the model:

```
print("Encoded new profile row sent to rf.predict():")
print(new_df.T.to_string(header=False))
print()
print("Company 'Google' in training classes?          ", 'Google' in
```

```

set(encoders['Company'].classes_)
print("Location 'Bengaluru, India' in training classes?", 'Bengaluru,
India' in set(encoders['Location'].classes_))
print("Fallback class used for unseen values (classes_[0]):",
encoders['Location'].classes_[0])

```

Encoded new profile row sent to rf.predict():

Job Title	12
Experience (Years)	5.0
Education	1
Skill Count	12
Industry	8
Company	7
Location	0
Certification	0
Remote_Hybrid	0
Remote_No	0
Remote_Yes	True

Company 'Google' in training classes? True
 Location 'Bengaluru, India' in training classes? False
 Fallback class used for unseen values (classes_[0]): Austin, USA

Figure 26: Supplementary check of the encoded new profile

Explanation

Profile: the dictionary new_profile describes a Machine Learning Engineer with 5.0 years of experience, a Masters degree, 12 skills, in the Technology industry at Google, located in “Bengaluru, India”, working remotely (Yes) and holding an “AWS ML Specialty” certification.

Preprocessing of the new profile: the profile is converted to a one-row DataFrame; Education is mapped with the same education_order; Remote Work is one-hot encoded; each label column is transformed with the encoder saved during training (encoders[col]); missing one-hot columns are added as 0 and the columns are re-ordered to match X.columns exactly. The model is then asked for a prediction with rf.predict(new_df)[0].

Result: the program printed **Predicted Salary: \$114,232.08** using the Random Forest model.

Technical clarifications about the notebook code (code kept unchanged):

- The comment in the code says unseen categories fall back to “the most frequent known label”, but the code actually uses `encoders[col].classes_[0]`, which is the **first label in alphabetical order**, not the most frequent one.
- The dataset contains the location “Bangalore, India”, but the profile uses “Bengaluru, India”, which is not in the training data. The supplementary check shows that this value was therefore replaced by the fallback “Austin, USA” (label 0) before prediction. The predicted salary above is hence for a profile whose Location was effectively “Austin, USA”. To predict for Bangalore, the spelling in `new_profile` should match the dataset (“Bangalore, India”).
- The prediction uses the Random Forest model (rf), as written in the notebook, although Gradient Boosting had the best scores in Program 12.

8. PROJECT WORKFLOW

The complete workflow of the project, from the dataset to the final prediction, is shown in the figure below.

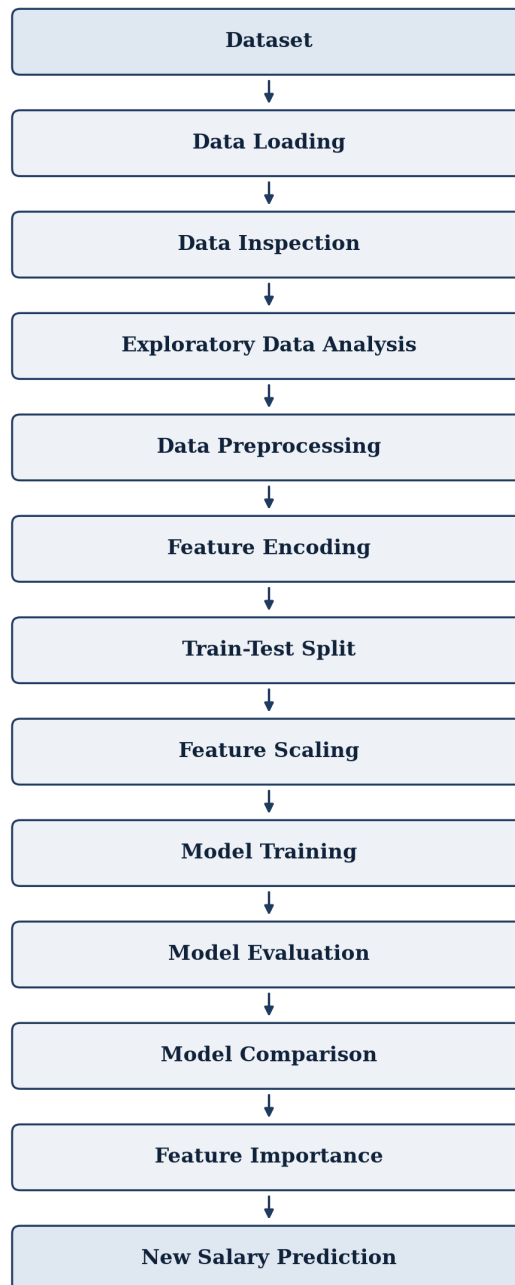


Figure 27: Complete Project Workflow

9. RESULTS AND DISCUSSION

All numbers in this chapter come from the program outputs of Chapter 7 (real execution), unless marked as a supplementary calculation from the dataset.

9.1 Dataset Observations

- The dataset has 500 records and 11 columns. A separate check found 0 duplicate rows, and the 166 “missing” Certification entries are the text “None” (no certification).
- Salary ranges from \$8,000 to \$216,300, with mean \$89,118.60 and median \$85,700; the distribution has two groups (Program 5.2).
- Experience and Skill Count have positive correlations with Salary (0.36 and 0.13).
- Median salary is lowest for Bachelors and highest for PhD; Remote Work type shows little difference.

The two salary groups seen in Programs 5.2 and 5.5 can be explained by the Location column. The following table is a **supplementary calculation** (a Pandas groupby on the dataset, not a notebook cell). Each of the six Indian cities has a median salary well below that of the other locations, which matches the lower band of the scatter plot and the first peak of the histogram.

Table 5: Salary by Location (supplementary groupby calculation on the dataset)

Location	Records	Median salary (USD)	Mean salary (USD)
Mumbai, India	33	36,200	35,209
Bangalore, India	30	37,100	37,220
Chennai, India	24	39,000	38,121
Pune, India	40	39,850	38,230
Hyderabad, India	34	40,450	38,535
Delhi, India	33	41,300	39,239
Remote	32	88,600	87,831
Berlin, Germany	34	104,300	104,668
Toronto, Canada	37	108,800	109,995
Singapore	26	111,100	120,081
London, UK	40	128,550	124,115
Seattle, USA	38	138,050	128,368
San Francisco, USA	32	140,050	136,788
New York, USA	38	140,050	140,966
Austin, USA	29	140,500	141,169

Consistent with this, Location has the highest feature importance in the Random Forest (0.5124).

9.2 Model Performance

Table 6: Model comparison (sorted by R²)

Model	MAE (USD)	RMSE (USD)	R ²
Gradient Boosting	9,090.10	12,646.38	0.9353
Random Forest	13,886.34	19,065.77	0.8529
Decision Tree	14,211.95	20,621.29	0.8280
Linear Regression	36,748.75	42,274.27	0.2770

- **MAE:** Gradient Boosting 9,090.10 < Random Forest 13,886.34 < Decision Tree 14,211.95 < Linear Regression 36,748.75. The Gradient Boosting MAE is about 10.0% of the mean test salary (\$90,806).

- **RMSE:** Gradient Boosting 12,646.38 < Random Forest 19,065.77 < Decision Tree 20,621.29 < Linear Regression 42,274.27.
- **R²:** Gradient Boosting 0.9353 > Random Forest 0.8529 > Decision Tree 0.8280 > Linear Regression 0.2770. Gradient Boosting explains about 93.5% of the salary variation in the test set, Linear Regression about 27.7%.

Since Gradient Boosting has the lowest MAE and RMSE and the highest R² on this test set, it is the best-performing of the four models **in this experiment**. The three tree-based models clearly outperform the Linear Regression baseline. A likely reason, not tested in the notebook, is that salary here depends on columns such as Location and Job Title in a non-linear way, while label-encoded numbers are arbitrary and a straight-line model cannot use them well; tree models can split on them.

9.3 Predicted vs Actual and Residuals

For Gradient Boosting, most points lie near the perfect-prediction line (Program 13). The residuals have a mean of -1,415.11 USD, which is small compared with the salary range, so the model shows little overall bias, and a standard deviation of 12,566.96 USD. 9 of 100 test predictions are more than 20,000 USD away from the actual salary, and one prediction is negative.

9.4 Feature Importance

In the Random Forest, Location (0.5124) and Experience (Years) (0.2304) are the most important features, followed by Job Title (0.1171). The Remote Work columns are the least important (each below 0.004). The notebook's concluding Markdown cell says that Experience and Skill Count tend to be the strongest numeric predictors and that Job Title, Industry and Location contribute meaningfully. The actual run supports this for Experience (the most important numeric feature), Job Title and Location. Skill Count (0.0445) is the second numeric feature but much less important, and Industry has a low importance (0.0232). This shows why conclusions should be checked against the actual results.

9.5 New Salary Prediction

For the sample profile, the Random Forest predicted **\$114,232.08**. Because the Location “Bengaluru, India” is not in the training data and was replaced by “Austin, USA”, this value should be read as a demonstration of the prediction pipeline rather than as a reliable estimate for Bengaluru.

9.6 Overall Discussion

The project successfully runs a complete regression workflow. The results are specific to this dataset and to one 80:20 split, and the best model was identified on the same test set that is used for reporting; therefore the numbers should be viewed as indicative of this experiment and not as a general claim about salaries.

10. ADVANTAGES

- Automated salary estimation for a given job profile.
- Uses several job-related factors together.
- Multiple regression models can be trained and compared with the same metrics.
- Visualisation (histograms, box plots, heatmap, scatter plots) helps in understanding patterns in the data.
- Feature importance shows which variables were most influential in the model.
- The notebook is simple to run (Google Colab) and can be extended with larger datasets.

11. LIMITATIONS

- The dataset size is limited (500 records, 100 for testing), so results can change with a different split.
- Predictions depend on the quality and coverage of the dataset; the source and collection method of the data are not documented in the repository.
- Label encoding gives arbitrary numbers to categories and may not capture real-world relationships.
- Salary depends on many external factors (market conditions, negotiation, company policy) that are not in the data.

- Predictions must not be treated as guaranteed salary offers.
- Model performance depends on the training data; unseen categories (such as a differently spelt location) are replaced by a fallback label.
- Only one train-test split is used; there is no cross-validation or hyperparameter tuning.

12. FUTURE ENHANCEMENTS

The following are suggestions for **future work**. They are **not implemented** in the current project.

- Use larger, real-world datasets and more job-market data.
- Use better categorical encoding (for example one-hot encoding or carefully validated target encoding).
- Apply hyperparameter tuning (for example GridSearchCV) and cross-validation.
- Try more advanced regression algorithms, such as XGBoost or LightGBM.
- Build a web-based prediction interface and deploy it with Flask or Streamlit.
- Use real-time salary data and cloud deployment.

13. CONCLUSION

In this project, a machine-learning system was developed to predict the salary of AI/ML-related jobs from the dataset `ai_ml_job_analysis.csv`. The data was inspected and explored with Pandas, Matplotlib and Seaborn, converted to numeric form using ordinal, one-hot and label encoding, split into training and testing sets and used to train four regression models.

The models were evaluated with MAE, RMSE and R^2 . In the actual run, Gradient Boosting gave the best scores on the test set ($R^2 = 0.9353$), Random Forest and Decision Tree performed similarly to each other, and Linear Regression performed the weakest ($R^2 = 0.2770$). Feature importance from the Random Forest showed Location and Experience as the most important features. A final cell shows how a new job profile is encoded in the same way as the training data and passed to the trained model for prediction.

The project demonstrates the practical machine-learning workflow for a regression problem. It also showed the importance of careful data handling, for example matching category spellings for new inputs. The results are limited to the given dataset and are not a guarantee of real salaries.

14. LEARNING OUTCOMES

- Python programming for data analysis.
- Using Pandas to load, inspect and transform datasets.
- Using NumPy for numerical operations.
- Data cleaning: checking missing values and understanding how they arise.
- Data preprocessing: ordinal, one-hot and label encoding, and scaling.
- Exploratory Data Analysis with statistics and graphs.
- Data visualisation with Matplotlib and Seaborn.
- Machine-learning concepts: supervised learning and regression.
- Training Linear Regression, Decision Tree, Random Forest and Gradient Boosting models.
- Model evaluation with MAE, RMSE and R^2 , and model comparison.
- Interpreting feature importance and residuals.
- Working with Jupyter Notebook / Google Colab.
- Understanding the practical AI/ML workflow from data to prediction.

15. VIVA QUESTIONS AND ANSWERS

Q1. What is Artificial Intelligence?

AI is making computers perform tasks that normally need human intelligence, such as learning, reasoning and decision-making.

Q2. What is Machine Learning?

Machine Learning is a part of AI in which computers learn patterns from data and make predictions without being explicitly programmed for every rule.

Q3. What is supervised learning?

Learning from data that has known answers (labels). The model learns the relation between the inputs and the correct output.

Q4. What is regression?

A supervised learning task in which the output is a continuous number, such as salary or price.

Q5. Why is this project a regression problem?

Because it predicts salary, which is a continuous numeric value, not a category.

Q6. What is the target variable?

The value we want to predict. Here it is Salary (USD).

Q7. What are features?

The input columns used for prediction, such as experience, education, skill count and location.

Q8. What is X?

X is the set of input features: all columns except Salary (USD). Its shape is (500, 11).

Q9. What is y?

y is the target: the Salary (USD) column, with shape (500,).

Q10. Why was train-test split used?

To train the model on one part of the data and test it on unseen data, which shows how well it will work on new cases.

Q11. Why was random_state=42 used?

To fix the random split so that the results are the same every time the code is run. 42 is just a commonly used seed.

Q12. What is Linear Regression?

An algorithm that fits a straight-line equation between the features and the target by minimising squared errors.

Q13. What is a Decision Tree?

A model that asks a series of yes/no questions on feature values to reach a prediction, like a flow chart.

Q14. What is Random Forest?

An ensemble of many decision trees trained on random parts of the data; the final prediction is the average of all trees.

Q15. What is Gradient Boosting?

An ensemble in which small trees are added one after another and each one corrects the errors of the previous ones.

Q16. What is MAE?

Mean Absolute Error: the average size of the prediction error, in dollars here. Lower is better.

Q17. What is RMSE?

Root Mean Squared Error: like MAE but large errors are punished more. Lower is better.

Q18. What is R²?

A score that tells how much of the variation in the target the model explains. 1 is perfect; higher is better.

Q19. What is StandardScaler?

A tool that rescales each feature to mean 0 and standard deviation 1.

Q20. What is LabelEncoder?

A tool that converts each distinct category into an integer label (0, 1, 2 ...).

Q21. What is One-Hot Encoding?

Converting a category into separate 0/1 columns, one for each category (for example Remote_Yes, Remote_No, Remote_Hybrid).

Q22. Why is categorical data encoded?

Machine-learning models work with numbers, so text categories must be converted to numeric form.

Q23. What is overfitting?

When a model learns the training data too closely (including noise) and performs poorly on new data.

Q24. What is feature importance?

A measure of how much each feature helped the model to make its predictions.

Q25. What is a residual?

Residual = Actual value – Predicted value; it is the error of one prediction.

Q26. Why is salary the target variable?

Because the aim of the project is to predict salary from the other job details.

Q27. Why are multiple models used?

To compare their performance on the same data and see which works best.

Q28. What is n_estimators?

The number of trees in a Random Forest, or the number of boosting stages in Gradient Boosting (300 in this project).

Q29. What is max_depth?

The maximum number of levels in a tree. It controls model complexity and helps limit overfitting.

Q30. What is learning_rate?

In Gradient Boosting, it controls how much each new tree contributes. A small value (0.05) means small, careful steps.

Q31. What does fit() do?

It trains the model (or scaler / encoder) by learning from the given data.

Q32. What does predict() do?

It uses the trained model to give predicted outputs for new input data.

Q33. Why is Pandas used?

To load the CSV file and to clean, inspect, encode and handle tabular data easily.

Q34. Why is NumPy used?

For fast numerical and array operations; Pandas and Scikit-learn are built on it.

Q35. Why are Matplotlib and Seaborn used?

To draw graphs such as histograms, heatmaps, box plots and scatter plots that help to understand the data and results.

16. REFERENCES

- [1] Python Software Foundation. (n.d.). *Python 3 documentation*.
<https://docs.python.org/3/> (accessed 1 October 2026).
- [2] The pandas development team. (n.d.). *pandas documentation*.
<https://pandas.pydata.org/docs/> (accessed 1 October 2026).
- [3] NumPy Developers. (n.d.). *NumPy documentation*. <https://numpy.org/doc/> (accessed 1 October 2026).
- [4] Scikit-learn developers. (n.d.). *scikit-learn: Machine learning in Python – user guide and API reference*. <https://scikit-learn.org/stable/> (accessed 1 October 2026).
- [5] The Matplotlib development team. (n.d.). *Matplotlib documentation*.
<https://matplotlib.org/stable/> (accessed 1 October 2026).
- [6] Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. Documentation: <https://seaborn.pydata.org/> (accessed 1 October 2026).
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [9] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
- [10] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [11] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).
- [12] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- [13] Google. (n.d.). *Google Colaboratory*. <https://colab.research.google.com/> (accessed 1 October 2026).
- [14] Vinod, V. (n.d.). *AI-ML-Salary-Prediction* [GitHub repository].
<https://github.com/Vismay-dev1/AI-ML-Salary-Prediction> (notebook: AI_ML_Salary_Prediction.ipynb; dataset: ai_ml_job_analysis.csv).

17. APPENDIX — COMPLETE SOURCE CODE

The complete code of the notebook AI_ML_Salary_Prediction.ipynb is given below in the notebook's order, one block per code cell, without modification. Each block is labelled with its program number from Chapter 7.

PROGRAM 1 — Load Dataset [notebook cell 2]

```
import os

# Upload dataset in Google Colab (skips automatically if file already
# present)
if not os.path.exists('ai_ml_job_analysis.csv'):
    try:
        from google.colab import files
        print("Please upload 'ai_ml_job_analysis.csv'")
        uploaded = files.upload()
    except ImportError:
        pass # not running in Colab

assert os.path.exists('ai_ml_job_analysis.csv'), "Dataset not found.
Please upload ai_ml_job_analysis.csv"
print("Dataset ready ✅")
```

PROGRAM 2 — Load and Display Data [notebook cell 3]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

df = pd.read_csv('ai_ml_job_analysis.csv')
df.head()
```

PROGRAM 3 — Dataset Shape and Information [notebook cell 5]

```
print("Shape:", df.shape)
df.info()
```

PROGRAM 5.1 — Statistical Summary [notebook cell 6]

```
df.describe(include='all').T
```

PROGRAM 4 — Check Missing Values [notebook cell 7]

```
# Missing values  
df.isnull().sum()
```

PROGRAM 5.2 — Salary Distribution [notebook cell 8]

```
# Target variable distribution  
plt.figure(figsize=(10,5))  
sns.histplot(df['Salary (USD)'], kde=True, bins=30, color='teal')  
plt.title('Salary Distribution')  
plt.xlabel('Salary (USD)')  
plt.show()
```

PROGRAM 5.3 — Correlation Analysis [notebook cell 9]

```
# Numeric feature correlations  
numeric_df = df.select_dtypes(include=[np.number]).drop(columns=['ID'])  
plt.figure(figsize=(6,5))  
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')  
plt.title('Correlation Heatmap (Numeric Features)')  
plt.show()
```

PROGRAM 5.4 — Salary by Categorical Features

[notebook cell 10]

```
# Salary by categorical features
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

sns.boxplot(data=df, x='Education', y='Salary (USD)', ax=axes[0,0])
axes[0,0].set_title('Salary by Education')

sns.boxplot(data=df, x='Remote Work', y='Salary (USD)', ax=axes[0,1])
axes[0,1].set_title('Salary by Remote Work Type')

top_titles = df['Job Title'].value_counts().index
sns.boxplot(data=df, x='Job Title', y='Salary (USD)', order=top_titles,
            ax=axes[1,0])
axes[1,0].set_title('Salary by Job Title')
axes[1,0].tick_params(axis='x', rotation=75)

sns.boxplot(data=df, x='Industry', y='Salary (USD)', ax=axes[1,1])
axes[1,1].set_title('Salary by Industry')
axes[1,1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

PROGRAM 5.5 — Experience vs Salary [notebook cell 11]

```
# Experience vs Salary
plt.figure(figsize=(9,5))
sns.scatterplot(data=df, x='Experience (Years)', y='Salary (USD)',
               hue='Education', alpha=0.7)
plt.title('Experience vs Salary')
plt.show()
```

PROGRAM 6 — Data Preprocessing [notebook cell 13]

```
from sklearn.preprocessing import LabelEncoder

data = df.drop(columns=['ID']).copy()

# Ordinal encode Education
education_order = {'Bachelors': 0, 'Masters': 1, 'PhD': 2}
data['Education'] = data['Education'].map(education_order)

# One-hot encode Remote Work
data = pd.get_dummies(data, columns=['Remote Work'], prefix='Remote')

# Label encode high-cardinality categorical columns
label_cols = ['Job Title', 'Industry', 'Company', 'Location',
              'Certification']
encoders = {}
for col in label_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col].astype(str))
    encoders[col] = le

data.head()
```

PROGRAM 7 — Feature and Target Separation [notebook cell 14]

```
X = data.drop(columns=['Salary (USD)'])
y = data['Salary (USD)']

print("Feature columns:", list(X.columns))
print("X shape:", X.shape, "| y shape:", y.shape)
```

PROGRAM 8 — Train-Test Split [notebook cell 16]

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
print(f"Train size: {X_train.shape[0]} | Test size: {X_test.shape[0]}")
```

PROGRAM 9 — Feature Scaling [notebook cell 17]

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

PROGRAMS 10 & 11 — Model Training and Evaluation

[notebook cell 19]

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
    GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score

results = {}

def evaluate(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    results[name] = {'MAE': mae, 'RMSE': rmse, 'R2': r2}
    print(f"{name:22s} | MAE: {mae:10,.2f} | RMSE: {rmse:10,.2f} | R2:
        {r2:.4f}")

# --- Linear Regression (scaled) ---
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
pred_lr = lr.predict(X_test_scaled)
evaluate('Linear Regression', y_test, pred_lr)

# --- Decision Tree ---
dt = DecisionTreeRegressor(max_depth=8, random_state=42)
dt.fit(X_train, y_train)
pred_dt = dt.predict(X_test)
evaluate('Decision Tree', y_test, pred_dt)

# --- Random Forest ---
rf = RandomForestRegressor(n_estimators=300, max_depth=12,
    random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
evaluate('Random Forest', y_test, pred_rf)

# --- Gradient Boosting ---
gb = GradientBoostingRegressor(n_estimators=300, learning_rate=0.05,
    max_depth=3, random_state=42)
gb.fit(X_train, y_train)
pred_gb = gb.predict(X_test)
evaluate('Gradient Boosting', y_test, pred_gb)
```

PROGRAM 12 — Model Comparison (table) [notebook cell 21]

```
results_df = pd.DataFrame(results).T.sort_values('R2', ascending=False)
results_df
```

PROGRAM 12 — Model Comparison (charts) [notebook cell 22]

```
fig, axes = plt.subplots(1, 2, figsize=(14,5))

results_df['R2'].plot(kind='bar', ax=axes[0], color='steelblue')
axes[0].set_title('R2 Score by Model')
axes[0].set_ylabel('R2')
axes[0].tick_params(axis='x', rotation=30)

results_df['RMSE'].plot(kind='bar', ax=axes[1], color='indianred')
axes[1].set_title('RMSE by Model')
axes[1].set_ylabel('RMSE (USD)')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```

PROGRAM 13 — Predicted vs Actual Graph [notebook cell 24]

```
best_model_name = results_df['R2'].idxmax()
print("Best model:", best_model_name)

preds_map = {
    'Linear Regression': pred_lr,
    'Decision Tree': pred_dt,
    'Random Forest': pred_rf,
    'Gradient Boosting': pred_gb,
}
best_preds = preds_map[best_model_name]

plt.figure(figsize=(7,7))
plt.scatter(y_test, best_preds, alpha=0.6, color='darkorange')
lims = [min(y_test.min(), best_preds.min()), max(y_test.max(),
    best_preds.max())]
plt.plot(lims, lims, 'k--', label='Perfect Prediction')
plt.xlabel('Actual Salary (USD)')
plt.ylabel('Predicted Salary (USD)')
plt.title(f'Predicted vs Actual Salary – {best_model_name}')
plt.legend()
plt.show()
```

PROGRAM 14 — Residual Analysis [notebook cell 25]

```
# Residual distribution for the best model
residuals = y_test.values - best_preds
plt.figure(figsize=(9,5))
sns.histplot(residuals, kde=True, bins=30, color='purple')
plt.title(f'Residual Distribution – {best_model_name}')
plt.xlabel('Residual (Actual - Predicted)')
plt.show()
```

PROGRAM 15 — Feature Importance [notebook cell 27]

```
importances = pd.Series(rf.feature_importances_,
    index=X.columns).sort_values(ascending=False)

plt.figure(figsize=(9,6))
sns.barplot(x=importances.values, y=importances.index, palette='viridis')
plt.title('Feature Importance – Random Forest')
plt.xlabel('Importance')
plt.show()

importances
```

PROGRAM 16 — New Salary Prediction [notebook cell 29]

```
new_profile = {
    'Job Title': 'Machine Learning Engineer',
    'Experience (Years)': 5.0,
    'Education': 'Masters',
    'Skill Count': 12,
    'Industry': 'Technology',
    'Company': 'Google',
    'Location': 'Bengaluru, India',
    'Remote Work': 'Yes',
    'Certification': 'AWS ML Specialty',
}

new_df = pd.DataFrame([new_profile])
new_df['Education'] = new_df['Education'].map(education_order)
new_df = pd.get_dummies(new_df, columns=['Remote Work'], prefix='Remote')

for col in label_cols:
    # unseen categories fall back to the most frequent known label
    known = set(encoders[col].classes_)
    new_df[col] = new_df[col].apply(lambda v: v if v in known else
                                    encoders[col].classes_[0])
    new_df[col] = encoders[col].transform(new_df[col].astype(str))

# align columns with training data (add any missing one-hot columns as 0)
for col in X.columns:
    if col not in new_df.columns:
        new_df[col] = 0
new_df = new_df[X.columns]

predicted_salary = rf.predict(new_df)[0]
print(f"Predicted Salary: ${predicted_salary:,.2f}")
```